CS3106L - Luit Lab 1 Report

Name: Shreyansh Kumar
Roll number: 240101094
Laboratory section: CSE

Q1 - Reading map

File	Role / Purpose	Type
`kernel/syscall.tbl`	Single source of truth mapping syscall numbers to names and handlers.	Edited
`kernel/syscallnums.h`	Defines the `SYS_<name>` integer constants for the kernel and user space.	Generated
`kernel/syscalltab.h`	Defines the function pointer array (`syscalls[]`) used by the kernel dispatcher.	Generated
`user/usys.S`	Contains the user-space assembly stubs that trigger the `ecall` instruction.	Generated

Q2 - Argument validation and syscall semantics

Using `atoi()` alone is insufficient because it silently handles invalid strings (e.g., returning 12 for "12x") and doesn't explicitly reject empty strings or completely non-numeric inputs, leading to unpredictable behaviour. Furthermore, if a negative value reaches `sys_sleep()` in the baseline kernel, it is cast to an unsigned integer. Due to two's complement representation, a negative number becomes an extremely large unsigned value. The kernel would then attempt to sleep for a massive number of ticks, causing the process to effectively hang forever.

Q3 - Pipe ownership

Descriptor table immediately after fork():

Process	Descriptor	Direction (Role)
Parent	`p2c[0]`	Read (Unused)
Parent	`p2c[1]`	Write (Used)
Parent	`c2p[0]`	Read (Used)
Parent	`c2p[1]`	Write (Unused)
Child	`p2c[0]`	Read (Used)
Child	`p2c[1]`	Write (Unused)
Child	`c2p[0]`	Read (Unused)
Child	`c2p[1]`	Write (Used)

Descriptor table after both processes close unused ends:

Process	Descriptor	Direction (Role)
Parent	`p2c[1]`	Write (Used)
Parent	`c2p[0]`	Read (Used)
Child	`p2c[0]`	Read (Used)
Child	`c2p[1]`	Write (Used)

Explanation: When a process forks, the child inherits a copy of all the parent's open file descriptors. A pipe reader will only receive an End-Of-File (EOF) signal from the OS when **every** write descriptor pointing to that pipe is closed across the entire system. If a process forgets to close an extra write end that it isn't using, the OS assumes that process might still write data in the future. As a result, the reading process will hang indefinitely waiting for more data, rather than correctly returning 0 (EOF) when the intended writer exits.

Q4 - Synchronisation without delays

The child cannot print "received ping" until its `read()` system call returns, which blocks until the parent executes its `write()` to the `p2c` pipe. This establishes the first happens-before relationship: the parent sends the ping *before* the child receives and prints it.

Similarly, the parent cannot print "received pong" until its `read()` system call returns, which blocks until the child executes its `write()` to the `c2p` pipe. Since the child performs this write *after* printing its line, the second happens-before relationship is established: the child prints its line *before* the parent receives the pong and prints its own line.

Adding `sleep()` would hide a design error by relying on arbitrary timing delays instead of deterministic data-flow synchronization. If the OS scheduler paused a process longer than the sleep duration, the output order would break. The blocking pipe reads ensure the correct sequence organically, regardless of process scheduling speeds.

Q5 - Constructing the process tree

Sorting the records by PID alone does not establish the hierarchical structure of a tree. It merely arranges the array linearly, which doesn't group parents with their children. A tree requires traversing the actual parent-child links.

In my representation, parent/child relationships are established by searching the snapshot array for processes where the `ppid` matches the current `root_pid`. Sibling ordering is guaranteed because the entire array is sorted by PID beforehand, meaning our search naturally finds children with lower PIDs first. Visited tracking is managed using a `visited[]` array indexed by the snapshot record's position, which prevents infinite recursion.

```
init(1) [sleep]
  sh(2) [sleep]
    pstree(10) [run]
  vdemo(5) [sleep]
    vdemo(6) [sleep]
    vdemo(7) [sleep]
    vdemo(8) [sleep]
    vdemo(9) [sleep]
```

Q6 - Snapshot consistency

A concrete sequence of events: Process A is scanned and recorded as having parent B. Right after A is scanned, but before B is scanned, parent B exits. Its process slot is immediately reused by a new process C. When the scan reaches that slot, it records process C instead of B. As a result, the snapshot shows A having parent B, but B does not exist in the table! Alternatively, rapid PID reuse could create a situation where A is recorded as the parent of B, and B is later recorded as the parent of A, forming an impossible cycle.

My program remains safe against missing parents/children because it independently verifies existence using `find_process_index()` and simply ignores missing links. It remains safe against cycles by marking each array index in a `visited[]` array before recursing; if it encounters an already-visited index, it prints a cycle warning and returns immediately, preventing an infinite loop.

Q7 - Generated ABI and fault injection

Generating both the user stubs and kernel dispatch tables from a single source of truth (`syscall.tbl`) ensures they are always perfectly synchronized. It prevents a class of ABI mismatch where a developer might add or renumber a system call in the kernel but forget to update the corresponding assembly stub in user-space, causing user programs to accidentally trigger the wrong kernel handler.

To test the checker, I deliberately modified `user/usys.S` by breaking the `exec` stub. When I ran the checker, it successfully detected the mismatch and reported: `ERROR: missing or mismatched SYSCALL exec, 7 stub in usys.S`. I then ran `python3 tools/gensyscalls.py`, which regenerated `usys.S` directly from `syscall.tbl`, overwriting my fault and fully repairing the ABI. The checker then correctly reported `ABI valid`.

Q8 - System-call path

1. **User-Space Stub**: The `sleep.c` program calls `sleep(5)`. This jumps to the generated assembly stub in `usys.S`, which loads the system call number 13 into the `a7` register, leaves the argument (5) in the `a0` register, and executes the `ecall` instruction.

2. **usertrap()**: The `ecall` triggers a trap into the kernel. The CPU saves the current Program Counter (PC) into the `sepc` CSR and sets the cause (`scause`) to 8 (Environment call from U-mode). The kernel's `usertrap()` function handles this trap.

3. **syscall()**: `usertrap()` recognises it's a system call and calls `syscall()`. To ensure the program resumes *after* the 4-byte `ecall` instruction when it returns to user-space, it advances the saved program counter (`epc`) by 4 bytes.

4. **Dispatch**: `syscall()` reads `a7` (which is 13) and looks it up in the `syscalls[]` dispatch array, jumping execution to `sys_sleep()`.

5. **Return**: After `sys_sleep()` finishes, it returns 0 for success. This return value is placed into the trap frame's `a0` register. Finally, `usertrapret()` prepares the CPU to return to user-space, restoring the modified `epc` and the new `a0` value.

Testing summary

Component	Tests performed	Result/evidence
sleep	Manual tests with 0, 5, -1, 12x, overflow values.	No output on valid; correct specific error messages on invalid input; no hangs.
pingpong	Repeated runs at `CPUS=1` and `CPUS=4`.	Exactly two ordered lines printed (`received ping`, `received pong`).
pstree	Default root, explicit root, absent root, active `vdemo`.	Tree printed with correct hierarchy, siblings sorted by PID, safely catches missing PIDs.
syscallmap.py	Valid files, deliberate mismatch in `usys.S`, regeneration.	Returned 0 on valid files; detected mismatch and threw error; repaired by `gensyscalls.py`.

Debugging reflection

Bug 1 (sleep.c): While implementing overflow protection in `sleep.c`, my initial attempt at handling overflow silently wrapped around to a negative number, causing sleep to throw a weird error. When I tried to fix it, my code failed to compile because I tried to use `INT_MAX` without importing the library, and I created a type mismatch between `main` and `parse_ticks`.

Evidence: The GCC compiler threw two errors. First, it complained that `INT_MAX` was undeclared and suggested adding the `limits.h` header. It also threw an incompatible pointer type error because `ticks` was a `long long` in main, but the helper function still expected an `int` pointer.

Correction: I included `limits.h` at the top of the file. I then updated the signature of the `parse_ticks` function to accept a `long long` pointer. This allowed the code to compile, and my overflow logic finally worked as intended.

Bug 2 (pstree.c): While implementing the process tree viewer, my initial code entered an infinite loop when testing it against the `vdemo` process. `vdemo` rapidly forks and exits, reusing PIDs and creating snapshot inconsistencies where two processes appeared to be each other's parents.

Evidence: When I ran `pstree` while `vdemo` was active in the background, the terminal hung completely and stopped responding. Adding print statements revealed that my recursive traversal was bouncing infinitely between two process indices.

Correction: I added a `visited` array to track which processes had already been printed during the recursive traversal. Before descending into a child, I checked if its index was already in the array. If it was, I returned immediately instead of recursing. This instantly fixed the infinite loop and made my tree traversal perfectly safe against cycles.